

Analytics Platform System Design with PostgreSQL

Table of Contents

1. [Overview](#)
 2. [Requirements](#)
 3. [Capacity Estimation](#)
 4. [Schema Design](#)
 5. [ASCII ER Diagram](#)
 6. [Indexing Strategy](#)
 7. [Query Patterns](#)
 8. [Scaling Strategy](#)
 9. [Tradeoffs](#)
 10. [Interview Discussion Points](#)
 11. [Common Interview Follow-ups](#)
 12. [Performance Considerations](#)
 13. [Cross-References](#)
-

Overview

An analytics platform ingests high-volume event streams (clicks, page views, conversions, errors), stores them durably, and serves aggregated metrics to dashboards and API consumers. The core design challenge is bridging two workloads that PostgreSQL handles differently: fast, concurrent writes (OLTP) and complex aggregation queries (OLAP). This document covers PostgreSQL-centric patterns: table partitioning, materialized views, BRIN indexes, TimescaleDB-style hypertables, and the transition point where a separate OLAP store becomes necessary.

Requirements

Functional Requirements

- Event ingestion: page views, clicks, conversions, errors, custom events
- Real-time dashboards: active users, revenue, error rates (lag < 30 seconds)
- Historical analysis: funnels, retention cohorts, trend analysis
- Retention policies: raw events for 90 days, aggregates for 2 years
- Alerting: notify when metric crosses threshold
- Report generation: CSV/JSON export of aggregates
- Segmentation: filter events by user properties, device, country

Non-Functional Requirements

- 100B+ events per day
 - Peak ingestion: 2M events per second
 - P99 dashboard query latency: < 2 seconds
 - P99 funnel query: < 10 seconds
 - Cardinality: 10M+ unique users, 500K+ unique event properties
 - Data retention: raw 90 days, aggregated 2 years, summary forever
 - 99.9% uptime for ingestion; 99% for queries
-

Capacity Estimation

Metric	Value
Events per day	100B
Peak ingest rate	2M events/sec
Average event size	500B
Daily raw data volume	50TB
Monthly raw data (90-day window)	1.5PB
Aggregated data (2 years)	~10TB
Dashboard queries per second	1K
Active dashboards	100K

Tiered Storage Plan

Tier	Data	Technology	Retention
Hot (real-time)	Last 24 hours	Redis / in-memory	24 hours
Warm (recent)	Last 90 days	PostgreSQL (partitioned)	90 days
Cold (archive)	90 days – 2 years	Columnar (Redshift, BigQuery, Clickhouse)	2 years
Summary	Aggregated forever	PostgreSQL materialized views	Indefinite

Note: At 100B events/day, PostgreSQL handles the ingestion pipeline and the warm tier. For OLAP queries on the cold tier, a dedicated columnar store is required. This document focuses on the PostgreSQL portion.

Schema Design

```
-- =====
-- EXTENSIONS
-- =====

CREATE EXTENSION IF NOT EXISTS "uuid-oss";
CREATE EXTENSION IF NOT EXISTS "btree_gin";
CREATE EXTENSION IF NOT EXISTS "pg_trgm";

-- =====
-- PROJECTS (Multi-tenant analytics)
-- =====

CREATE TABLE projects (
  project_id      UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
  name            VARCHAR(300)  NOT NULL,
  api_key         VARCHAR(64)   NOT NULL,
  org_id          UUID          NOT NULL,
  timezone        VARCHAR(60)   NOT NULL DEFAULT 'UTC',
```

```

    is_active          BOOLEAN          NOT NULL DEFAULT TRUE,
    created_at         TIMESTAMPTZ      NOT NULL DEFAULT now(),
    CONSTRAINT uq_api_key UNIQUE (api_key)
);

-- =====
-- EVENTS (Core Ingestion Table – Partitioned)
-- =====
CREATE TABLE events (
    event_id           UUID              NOT NULL DEFAULT uuid_generate_v4(),
    project_id         UUID              NOT NULL REFERENCES projects(project_id),
    event_name         VARCHAR(200)     NOT NULL,
    occurred_at        TIMESTAMPTZ      NOT NULL,
    received_at        TIMESTAMPTZ      NOT NULL DEFAULT now(),
    -- User identity
    user_id            VARCHAR(200),    -- application user ID (not DB UUID)
    anonymous_id        VARCHAR(200),    -- pre-login ID
    session_id         VARCHAR(200),
    -- Context
    country_code       CHAR(2),
    city               VARCHAR(100),
    device_type        VARCHAR(20),     -- 'desktop', 'mobile', 'tablet'
    os                 VARCHAR(50),
    browser            VARCHAR(50),
    app_version        VARCHAR(50),
    -- Properties (flexible schema)
    properties         JSONB            NOT NULL DEFAULT '{}',
    -- Revenue
    revenue            NUMERIC(14,4),
    currency           CHAR(3),
    -- Processing state
    is_processed       BOOLEAN          NOT NULL DEFAULT FALSE,
    PRIMARY KEY (event_id, occurred_at)
) PARTITION BY RANGE (occurred_at);

-- Daily partitions for fast access and lifecycle management
CREATE TABLE events_2024_01_01 PARTITION OF events
    FOR VALUES FROM ('2024-01-01') TO ('2024-01-02');
-- pg_partman automates partition creation/deletion

-- =====
-- USERS (Identified users for analytics)
-- =====
CREATE TABLE analytics_users (
    project_id         UUID              NOT NULL REFERENCES projects(project_id),
    user_id            VARCHAR(200)     NOT NULL,
    anonymous_id        VARCHAR(200),
    first_seen_at      TIMESTAMPTZ      NOT NULL,
    last_seen_at       TIMESTAMPTZ      NOT NULL,
    event_count        BIGINT           NOT NULL DEFAULT 0,
    traits             JSONB            NOT NULL DEFAULT '{}', -- name, email, plan,
    etc.

```

```

    PRIMARY KEY (project_id, user_id)
);

-- =====
-- EVENT METADATA (Catalog of known event schemas)
-- =====

CREATE TABLE event_definitions (
    project_id      UUID          NOT NULL REFERENCES projects(project_id),
    event_name      VARCHAR(200)  NOT NULL,
    description     TEXT,
    event_count     BIGINT        NOT NULL DEFAULT 0,
    last_seen_at    TIMESTAMPTZ,
    property_schema JSONB,        -- {"url": "string", "revenue": "number", ...}
    PRIMARY KEY (project_id, event_name)
);

-- =====
-- AGGREGATED METRICS (Pre-computed – Fast Dashboard Reads)
-- =====

-- Hourly event counts per project
CREATE TABLE event_counts_hourly (
    project_id      UUID          NOT NULL REFERENCES projects(project_id),
    event_name      VARCHAR(200)  NOT NULL,
    hour_start      TIMESTAMPTZ   NOT NULL,
    total_events    BIGINT        NOT NULL DEFAULT 0,
    unique_users    BIGINT        NOT NULL DEFAULT 0,
    total_revenue   NUMERIC(18,4) NOT NULL DEFAULT 0,
    PRIMARY KEY (project_id, event_name, hour_start)
) PARTITION BY RANGE (hour_start);

-- Daily event counts (retained 2 years)
CREATE TABLE event_counts_daily (
    project_id      UUID          NOT NULL REFERENCES projects(project_id),
    event_name      VARCHAR(200)  NOT NULL,
    stat_date       DATE          NOT NULL,
    total_events    BIGINT        NOT NULL DEFAULT 0,
    unique_users    BIGINT        NOT NULL DEFAULT 0,
    total_revenue   NUMERIC(18,4) NOT NULL DEFAULT 0,
    -- Breakdowns
    by_country      JSONB,        -- {"US": 1200, "GB": 300, ...}
    by_device       JSONB,
    by_browser      JSONB,
    PRIMARY KEY (project_id, event_name, stat_date)
) PARTITION BY RANGE (stat_date);

-- =====
-- FUNNELS
-- =====

CREATE TABLE funnels (
    funnel_id       UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id      UUID          NOT NULL REFERENCES projects(project_id),

```

```

name          VARCHAR(300)    NOT NULL,
steps         JSONB           NOT NULL,
-- [{"event": "page_view", "filters": {"url": "/pricing"}},
-- {"event": "signup_start", "filters": {}},
-- {"event": "signup_complete", "filters": {}}]
conversion_window_hours INT     NOT NULL DEFAULT 24,
created_at    TIMESTAMPTZ      NOT NULL DEFAULT now()
);

-- Pre-computed funnel results (refreshed hourly by background job)
CREATE TABLE funnel_results (
    funnel_id      UUID          NOT NULL REFERENCES funnels(funnel_id),
    computed_date  DATE          NOT NULL,
    step_number    SMALLINT      NOT NULL,
    event_name     VARCHAR(200)  NOT NULL,
    user_count     BIGINT        NOT NULL,
    conversion_rate NUMERIC(7,4),
    avg_time_to_step_seconds BIGINT,
    PRIMARY KEY (funnel_id, computed_date, step_number)
);

-- =====
-- RETENTION COHORTS
-- =====
CREATE TABLE retention_cohorts (
    project_id     UUID          NOT NULL REFERENCES projects(project_id),
    cohort_date    DATE          NOT NULL, -- week/month when users first
appeared
    day_offset     INT           NOT NULL, -- days since cohort date
    cohort_size    INT           NOT NULL,
    retained_users INT           NOT NULL,
    retention_rate  NUMERIC(7,4) GENERATED ALWAYS AS
        (retained_users::NUMERIC / NULLIF(cohort_size, 0)) STORED,
    PRIMARY KEY (project_id, cohort_date, day_offset)
);

-- =====
-- ALERTS
-- =====
CREATE TABLE alert_definitions (
    alert_id       UUID          PRIMARY KEY DEFAULT uuid_generate_v4(),
    project_id     UUID          NOT NULL REFERENCES projects(project_id),
    name           VARCHAR(300)  NOT NULL,
    event_name     VARCHAR(200)  NOT NULL,
    metric         VARCHAR(50)   NOT NULL, --
'count', 'unique_users', 'revenue', 'error_rate'
    operator       VARCHAR(10)   NOT NULL, -- '>', '<', '>=', '<=', '='
    threshold      NUMERIC(20,4) NOT NULL,
    window_minutes INT           NOT NULL DEFAULT 60,
    is_active      BOOLEAN       NOT NULL DEFAULT TRUE,
    last_triggered_at TIMESTAMPTZ,
    created_at     TIMESTAMPTZ   NOT NULL DEFAULT now()
);

```

```

);

CREATE TABLE alert_incidents (
    incident_id    UUID                PRIMARY KEY DEFAULT uuid_generate_v4(),
    alert_id       UUID                NOT NULL REFERENCES alert_definitions(alert_id),
    triggered_at   TIMESTAMPTZ         NOT NULL DEFAULT now(),
    resolved_at    TIMESTAMPTZ,
    metric_value   NUMERIC(20,4)       NOT NULL,
    threshold      NUMERIC(20,4)       NOT NULL,
    notified       BOOLEAN              NOT NULL DEFAULT FALSE
);

-- =====
-- MATERIALIZED VIEWS FOR DASHBOARDS
-- =====

-- 7-day event summary per project (refreshed every hour)
CREATE MATERIALIZED VIEW project_weekly_summary AS
SELECT
    project_id,
    event_name,
    SUM(total_events) AS total_7d,
    SUM(unique_users) AS unique_users_7d,
    SUM(total_revenue) AS revenue_7d,
    MAX(total_events) AS peak_hour_events
FROM event_counts_hourly
WHERE hour_start >= now() - INTERVAL '7 days'
GROUP BY project_id, event_name;

CREATE UNIQUE INDEX ON project_weekly_summary (project_id, event_name);

-- Real-time active users (last 5 minutes – refreshed every minute)
CREATE MATERIALIZED VIEW active_users_realtime AS
SELECT
    project_id,
    COUNT(DISTINCT user_id) AS active_users,
    COUNT(*) AS events_last_5min
FROM events
WHERE occurred_at >= now() - INTERVAL '5 minutes'
GROUP BY project_id;

CREATE UNIQUE INDEX ON active_users_realtime (project_id);

-- Refresh schedule (production: run via cron or pg_cron extension)
-- SELECT cron.schedule('refresh_weekly_summary', '5 * * * *',
--     'REFRESH MATERIALIZED VIEW CONCURRENTLY project_weekly_summary');

```

ASCII ER Diagram

```

+-----+      +-----+      +-----+
| projects |1-----N| events |      |event_defs |
+-----+      +-----+      +-----+
      |1          |
      |N          |

+-----+      +-----+
|analytics |      |event_counts_hourly|
|  users   |      +-----+
+-----+      +-----+
                        |event_counts_daily |
                        +-----+

+-----+      +-----+      +-----+
| projects |1-----N| funnels |1-----N|funnel_rslt|
+-----+      +-----+      +-----+

+-----+      +-----+
| projects |1-----N| retention |
+-----+      | cohorts |
                        +-----+

+-----+      +-----+      +-----+
| projects |1-----N|alert_definition|1-----N| alert_incidents |
+-----+      +-----+      +-----+

```

MATERIALIZED VIEWS:

```

+-----+      +-----+
|project_weekly_summ |      | active_users_realtime|
+-----+      +-----+

```

Indexing Strategy

```

-- Events: the primary ingestion table – indexes must be minimal
-- Too many indexes on a high-write table cause write slowdown

-- 1. Partition key (occurred_at) is implicit
-- 2. Project + time range queries (most common)
CREATE INDEX idx_events_project_time ON events (project_id, occurred_at DESC);

-- 3. Event name filter (most dashboards filter by event type)
CREATE INDEX idx_events_name_time ON events (project_id, event_name, occurred_at
DESC);

-- 4. JSONB properties – only if specific property is frequently filtered
CREATE INDEX idx_events_properties ON events USING GIN (properties)
WHERE properties != '{}';

-- 5. BRIN index for occurred_at (very small, leverages physical ordering)
CREATE INDEX idx_events_brin_time ON events USING BRIN (occurred_at)
WITH (pages_per_range = 128);

```

```

-- Analytics users: identity resolution
CREATE INDEX idx_agg_users_project ON analytics_users (project_id, user_id);
CREATE INDEX idx_agg_users_anon ON analytics_users (project_id, anonymous_id)
    WHERE anonymous_id IS NOT NULL;

-- Hourly counts: dashboard line chart
CREATE INDEX idx_counts_hourly ON event_counts_hourly
    (project_id, event_name, hour_start DESC);

-- Daily counts: trend analysis
CREATE INDEX idx_counts_daily ON event_counts_daily
    (project_id, event_name, stat_date DESC);

-- Alerts: evaluation job
CREATE INDEX idx_alert_active ON alert_definitions (project_id, is_active)
    WHERE is_active = TRUE;

-- Funnel results: dashboard
CREATE INDEX idx_funnel_results ON funnel_results (funnel_id, computed_date
DESC);

```

Query Patterns

Q1 — Real-Time Dashboard: Active Users (last 5 min)

```

-- Served from materialized view (refreshed every 60 seconds)
SELECT active_users, events_last_5min
FROM active_users_realtime
WHERE project_id = $1;

```

Q2 — Event Volume Over Time (hourly, 7 days)

```

SELECT
    DATE_TRUNC('hour', h.hour_start) AS hour,
    h.total_events,
    h.unique_users,
    h.total_revenue
FROM event_counts_hourly h
WHERE h.project_id = $1
    AND h.event_name = $2
    AND h.hour_start >= now() - INTERVAL '7 days'
ORDER BY h.hour_start ASC;

```

Q3 — Daily Totals with Country Breakdown

```

SELECT
    d.stat_date,

```

```

        d.total_events,
        d.unique_users,
        d.total_revenue,
        jsonb_each_text(d.by_country) AS country_breakdown
FROM event_counts_daily d
WHERE d.project_id = $1
      AND d.event_name = $2
      AND d.stat_date BETWEEN $from_date AND $to_date
ORDER BY d.stat_date ASC;

```

Q4 — Hourly Aggregation Job (runs every 5 minutes)

```

INSERT INTO event_counts_hourly
    (project_id, event_name, hour_start, total_events, unique_users, total_revenue)
SELECT
    e.project_id,
    e.event_name,
    DATE_TRUNC('hour', e.occurred_at) AS hour_start,
    COUNT(*) AS total,
    COUNT(DISTINCT COALESCE(e.user_id, e.anonymous_id)) AS uniq,
    COALESCE(SUM(e.revenue), 0) AS revenue
FROM events e
WHERE e.occurred_at >= DATE_TRUNC('hour', now()) - INTERVAL '1 hour'
      AND e.occurred_at < DATE_TRUNC('hour', now())
GROUP BY e.project_id, e.event_name, DATE_TRUNC('hour', e.occurred_at)
ON CONFLICT (project_id, event_name, hour_start)
DO UPDATE SET
    total_events = EXCLUDED.total_events,
    unique_users = EXCLUDED.unique_users,
    total_revenue = EXCLUDED.total_revenue;

```

Q5 — Funnel Computation (on-demand or scheduled)

```

-- 3-step funnel: viewed_pricing → started_trial → converted
WITH step_1 AS (
    SELECT DISTINCT COALESCE(user_id, anonymous_id) AS uid, MIN(occurred_at) AS t1
    FROM events
    WHERE project_id = $project_id AND event_name = 'viewed_pricing'
      AND occurred_at BETWEEN $start AND $end
    GROUP BY 1
),
step_2 AS (
    SELECT DISTINCT e.user_id AS uid, MIN(e.occurred_at) AS t2
    FROM events e
    JOIN step_1 s ON s.uid = COALESCE(e.user_id, e.anonymous_id)
    WHERE e.project_id = $project_id AND e.event_name = 'started_trial'
      AND e.occurred_at > s.t1
      AND e.occurred_at <= s.t1 + INTERVAL '24 hours'
    GROUP BY e.user_id

```

```

),
step_3 AS (
    SELECT DISTINCT e.user_id AS uid
    FROM events e
    JOIN step_2 s ON s.uid = e.user_id
    WHERE e.project_id = $project_id AND e.event_name = 'converted'
        AND e.occurred_at > s.t2
        AND e.occurred_at <= s.t2 + INTERVAL '24 hours'
)
SELECT
    (SELECT COUNT(*) FROM step_1) AS step_1_users,
    (SELECT COUNT(*) FROM step_2) AS step_2_users,
    (SELECT COUNT(*) FROM step_3) AS step_3_users,
    ROUND((SELECT COUNT(*) FROM step_2)::NUMERIC /
        NULLIF((SELECT COUNT(*) FROM step_1), 0) * 100, 2) AS step_1_to_2_pct,
    ROUND((SELECT COUNT(*) FROM step_3)::NUMERIC /
        NULLIF((SELECT COUNT(*) FROM step_2), 0) * 100, 2) AS step_2_to_3_pct;

```

Q6 — Retention Cohort Computation (weekly cohorts)

```

-- Cohort: users who first appeared in week of $cohort_start
-- Retained: users who also appeared in week of $cohort_start + N weeks
WITH cohort_users AS (
    SELECT DISTINCT COALESCE(user_id, anonymous_id) AS uid
    FROM events
    WHERE project_id = $project_id
        AND occurred_at >= $cohort_start
        AND occurred_at < $cohort_start + INTERVAL '7 days'
),
retained AS (
    SELECT
        $week_offset AS offset,
        COUNT(DISTINCT COALESCE(e.user_id, e.anonymous_id)) AS retained
    FROM events e
    JOIN cohort_users c ON c.uid = COALESCE(e.user_id, e.anonymous_id)
    WHERE e.project_id = $project_id
        AND e.occurred_at >= $cohort_start + ($week_offset || ' weeks')::INTERVAL
        AND e.occurred_at < $cohort_start + (($week_offset + 1) || ' weeks')::INTERVAL
)
INSERT INTO retention_cohorts
    (project_id, cohort_date, day_offset, cohort_size, retained_users)
SELECT
    $project_id, $cohort_start::DATE, $week_offset * 7,
    (SELECT COUNT(*) FROM cohort_users),
    (SELECT retained FROM retained)
ON CONFLICT (project_id, cohort_date, day_offset)
DO UPDATE SET retained_users = EXCLUDED.retained_users;

```

Q7 — Alert Evaluation (runs every 5 minutes)

```

SELECT
  a.alert_id, a.name, a.event_name, a.metric, a.operator, a.threshold,
  CASE a.metric
    WHEN 'count' THEN (
      SELECT COUNT(*) FROM events
      WHERE project_id = a.project_id
        AND event_name = a.event_name
        AND occurred_at >= now() - (a.window_minutes || ' minutes')::INTERVAL
    )
    WHEN 'unique_users' THEN (
      SELECT COUNT(DISTINCT COALESCE(user_id, anonymous_id)) FROM events
      WHERE project_id = a.project_id
        AND event_name = a.event_name
        AND occurred_at >= now() - (a.window_minutes || ' minutes')::INTERVAL
    )
    WHEN 'revenue' THEN (
      SELECT COALESCE(SUM(revenue), 0) FROM events
      WHERE project_id = a.project_id
        AND event_name = a.event_name
        AND occurred_at >= now() - (a.window_minutes || ' minutes')::INTERVAL
    )
  END AS current_value
FROM alert_definitions a
WHERE a.is_active = TRUE
  AND (a.last_triggered_at IS NULL OR a.last_triggered_at < now() - INTERVAL '15
minutes');

```

Q8 — Raw Event Query with Property Filter

```

SELECT
  e.event_id, e.user_id, e.session_id,
  e.event_name, e.occurred_at,
  e.country_code, e.device_type,
  e.properties->>'url' AS url,
  e.properties->>'referrer' AS referrer,
  e.revenue
FROM events e
WHERE e.project_id = $1
  AND e.event_name = 'page_view'
  AND e.occurred_at BETWEEN $from AND $to -- partition pruning
  AND e.country_code = $country
  AND e.properties->>'url' LIKE '/pricing%'
ORDER BY e.occurred_at DESC
LIMIT 1000;

```

Q9 — Session Analysis: Events Per Session

```

SELECT
  session_id,

```

```

    MIN(occurred_at) AS session_start,
    MAX(occurred_at) AS session_end,
    EXTRACT(EPOCH FROM MAX(occurred_at) - MIN(occurred_at)) AS duration_seconds,
    COUNT(*) AS event_count,
    COUNT(*) FILTER (WHERE event_name = 'page_view') AS page_views,
    bool_or(event_name = 'purchase') AS converted,
    MAX(CASE WHEN event_name = 'purchase' THEN revenue END) AS revenue
FROM events
WHERE project_id = $1
    AND session_id IS NOT NULL
    AND occurred_at BETWEEN $from AND $to
GROUP BY session_id
ORDER BY session_start DESC
LIMIT 100;

```

Q10 — Partition Maintenance (scheduled)

```

-- Drop partitions older than 90 days
DO $$
DECLARE
    v_partition TEXT;
BEGIN
    FOR v_partition IN
        SELECT tablename FROM pg_tables
        WHERE tablename LIKE 'events_%'
            AND to_date(substring(tablename, 8), 'YYYY_MM_DD') < CURRENT_DATE - 90
    LOOP
        EXECUTE 'DROP TABLE IF EXISTS ' || v_partition;
        RAISE NOTICE 'Dropped partition: %', v_partition;
    END LOOP;
END $$;

```

Scaling Strategy

Ingestion Pipeline

At 2M events/second, no single PostgreSQL instance can sustain direct INSERTs. Use a buffering pipeline:

```

Client SDK
|
| (batched HTTP, every 1 second)
v
Ingestion Service (stateless, N instances)
|
+-- Kafka topic 'raw_events' (partitioned by project_id)
|
+-- Consumer A: Buffer 1000 events, COPY to PostgreSQL
|   (sustains ~500K rows/sec per PostgreSQL node)
|
+-- Consumer B: Update Redis counters for real-time dashboards

```

```
| (INCR project:{id}:event:{name}:count:YYYYMMHH)
|
+-- Consumer C: Write to data warehouse (Redshift/BigQuery)
    (full fidelity, no retention limit)
```

Phase 1: Single PostgreSQL (0–10B events/day)

- Daily partitions, retain 90 days
- Hourly aggregation job
- Materialized views for dashboards
- Redis counters for real-time 5-minute windows

Phase 2: Partitioned + Read Replicas (10B–100B events/day)

- Partitioned events table + 2 read replicas
- All dashboard queries on read replicas
- Kafka consumer pool for parallel ingestion
- Separate PostgreSQL for aggregates (no competition with raw events)

Phase 3: Hybrid OLTP + OLAP (100B+ events/day)

- PostgreSQL: last 7 days of raw events (fast queries, fits in RAM)
- Redshift/BigQuery/ClickHouse: historical raw events (columnar, OLAP-optimized)
- PostgreSQL: aggregates table (fast dashboard reads)
- Real-time metrics: Redis (counters, sorted sets for leaderboards)
- Kafka: connects all layers

ClickHouse Alternative for Hot Tier

ClickHouse is purpose-built for analytics: column-store, compressed, vectorized execution. For 100B events/day, consider:

- ClickHouse for `events` table (10–100x faster aggregation than PostgreSQL)
- PostgreSQL for `projects`, `funnels`, `alerts`, `retention_cohorts` (metadata)
- Data flows: `Kafka` → `ClickHouse` + `PostgreSQL` (fan-out)

Tradeoffs

Decision	Chosen	Alternative	Why
Daily event partitions	Daily	Hourly	Daily is manageable (90 partitions active); hourly would require 2,160 partitions
BRIN on <code>occurred_at</code>	BRIN + B-Tree	B-Tree only	BRIN is 1000x smaller for append-only time-series; B-Tree needed for point lookups
Materialized views for dashboards	Precomputed	On-demand aggregation	Funnel queries on 100B events take minutes; pre-computed views take milliseconds

JSONB for event properties	Flexible schema	Fixed columns	Event schemas change constantly (new properties); flexible schema avoids migrations
Kafka buffer before PostgreSQL	Kafka + COPY	Direct INSERT	Direct INSERT at 2M/sec would overwhelm PostgreSQL; COPY is 10x faster than INSERT

Interview Discussion Points

- 1. Why not just use PostgreSQL for everything?** PostgreSQL handles the warm tier (last 90 days) well. For historical OLAP queries (funnels across 2 years of data), columnar stores (Redshift, BigQuery, ClickHouse) are 10–100x faster because they only read the columns needed (I/O reduction) and are optimized for vectorized aggregation. PostgreSQL reads all columns even for `SELECT COUNT(*)`.
- 2. How does retention cohort analysis work?** You define a cohort (users who signed up in week W). For each subsequent week W+N, you count how many of those users were active. The result is a triangle matrix showing week-over-week retention. This requires two self-joins on the events table, which is expensive at scale — pre-compute and store in `retention_cohorts`.
- 3. How do you handle late-arriving events?** Events can arrive minutes or hours late (mobile offline). The `occurred_at` column (event time) is the analytical timestamp; `received_at` (server time) is used for partition routing. The aggregation job runs with a 2-hour lag to catch late arrivals, then runs again to update the final hourly counts.
- 4. Why use BRIN indexes for time-series data?** BRIN (Block Range INdex) stores min/max values for each range of disk pages. For append-only time-series data, pages are naturally ordered by time — BRIN efficiently prunes pages for time-range queries. A BRIN index is 1,000x smaller than a B-Tree index on the same column, reducing I/O for table scans.

Common Interview Follow-ups

Q: How do you compute unique user counts efficiently at scale? A: `COUNT(DISTINCT user_id)` is expensive at billions of rows (requires sorting and deduplication). Use HyperLogLog (HLL) for approximate counts: PostgreSQL has the `hll` extension. Store `hll_add_agg(hll_hash_text(user_id))` per hour/day. Merge HLL sketches for larger windows: `hll_cardinality(hll_union_agg(hll_sketch))`. Error rate: ~1%, which is acceptable for analytics.

Q: How do you handle a user deleting their data (GDPR right to erasure)? A: Set `user_id = NULL` on all events for that user (backfill). Historical aggregates that included this user remain — they cannot be retroactively corrected for aggregated data. Raw events with NULL `user_id` are effectively anonymous. The user record in `analytics_users` is hard deleted.

Q: How do you implement real-time dashboards updating every 5 seconds? A: Don't query PostgreSQL every 5 seconds — cache the result. Use Redis counters (`INCR project:{id}:event:{name}:2024010115`) updated by the Kafka consumer on every event. Dashboard API reads from Redis (sub-millisecond). PostgreSQL stores the authoritative counts; Redis is the cache. Refresh PostgreSQL from Redis every minute.

Performance Considerations

- **COPY vs. INSERT:** Bulk ingest via `COPY` is 5–10x faster than multi-row `INSERT`. Buffer events in the Kafka consumer and `COPY` every 10,000 events or every 5 seconds.
 - **Partition pruning is critical:** All analytical queries MUST include `occurred_at BETWEEN $from AND $to` . Without this, PostgreSQL scans all 90 daily partitions. Explain plans should show "Append (cost=... → partitions pruned: 88 of 90)".
 - **Parallel query:** Enable parallel workers for aggregation queries on large partitions: `SET max_parallel_workers_per_gather = 8` . Funnel queries can utilize 8 cores, reducing runtime from 60s to 10s.
 - **work_mem for funnel queries:** Funnel CTEs sort millions of rows. Set `SET work_mem = '1GB'` for the session running funnel computation to prevent disk spills.
 - **VACUUM on events partitions:** Append-only workload means almost no dead tuples. Disable autovacuum on individual event partitions: `ALTER TABLE events_2024_01_01 SET (autovacuum_enabled = FALSE)` .
-

Cross-References

- See `18_Architecture_Case_Studies/05_saas_product.md` for usage metering patterns
- See `01_url_shortener.md` for click analytics pipeline
- See `08_system_design_framework.md` for interview presentation framework
- PostgreSQL partitioning: Chapter 5 of this guide
- Materialized views: Chapter 7 of this guide
- BRIN indexes: Chapter 8 of this guide
- Full-text search: Chapter 9 of this guide